



数据结构

教师: 刘芳岑

邮箱: liufc@hainanu.edu.cn

学院: 计算机科学与技术学院

时间: 2026春季学期

➤ 栈（逻辑结构）：

是一种运算受限的线性表，只能在一端进行插入或删除操作的线性表
先进后出（死胡同）

6个基本运算（重点Pop, Push）

➤ 顺序栈：四要素：

（物理结构）栈空条件、栈满条件

进栈e操作：先++；后存放

退栈操作：先取值，后--

➤ 链栈：四要素：

（物理结构）栈空条件、栈满条件（不考虑）

进栈e操作：放在头结点之后

退栈操作

➤ 共享栈

➤ 简单表达式求值：中缀表达式转后缀表达式；后缀表达式求值

由两个栈共享一个数组空间的好处是 ()

- A. 减少存取时间, 降低上溢出发生的概率
- B. 节省存储空间, 降低上溢出发生的概率
- C. 减少存取时间, 降低下溢出发生的概率
- D. 节省存储空间, 降低下溢出发生的概率

3.1 栈

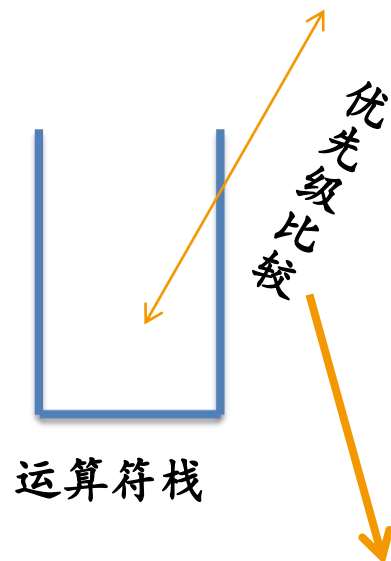
3.1.4 栈的应用

中缀表达式转后缀表达式

$\text{exp} \Rightarrow \text{postexp}$

◆ 情况1 (没有括号)

$\text{exp} =$ “1 + 2 + 3”



$\text{postexp}:$

“1+2+3” $\Rightarrow$ “1 2 + 3 +”

- 优先级相同：先进栈的先退栈即先执行。只有大于栈顶优先级才能直接进栈
- exp 扫描完毕，所有运算符退栈

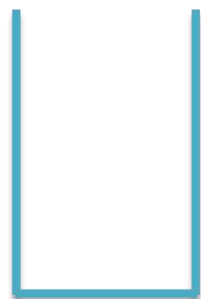
3.1 栈

3.1.4 栈的应用

exp $\Rightarrow$ postexp

◆ 情况2 (带有括号)

exp= “ 2 * (1 + 3) - 4 ”



运算符栈

postexp:



postexp = “2 1 3 + * 4 -”

- 开始时, 任何运算符都进栈
- (: 一个子表达式开始, 进栈
- 栈顶为(: 任何运算符进栈
-): 退栈到 (
- 只有大于栈顶的优先级, 才进栈; 否则退栈

3.1 栈

3.1.4 栈的应用

中缀表达式: **$10 * (15 + 3) - 6$**

后缀表达式: **$10\ 15\ 3\ +\ *\ 6\ -$**



3.1 栈

3.1.4 后缀表达式求值

规则:遇到数字就进栈,遇到符号将处于栈顶两个数字出栈,进行运算,并将结果进栈,直至最终获得结果。

后缀表达式:**9 3 1 - 3 * + 10 2 / +**

运算结果:**20**

第3章 栈和队列

提纲

CONTENTS

3.1 栈



3.2 队列

3.2 队 列

3.2.1 队列的定义

队列简称队，它也是一种运算受限的线性表。



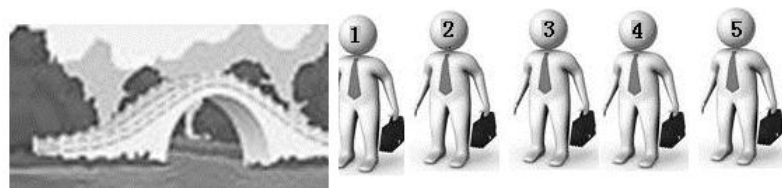
队列只能选取一个端点进行插入操作，另一个端点进行删除操作

3.2 队列

3.2.1 队列的定义

队列的主要特点是**先进先出**，所以又把队列称为**先进先出表**。

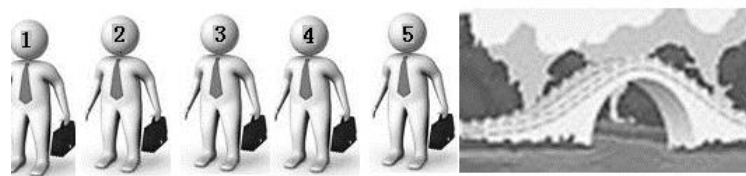
例如：



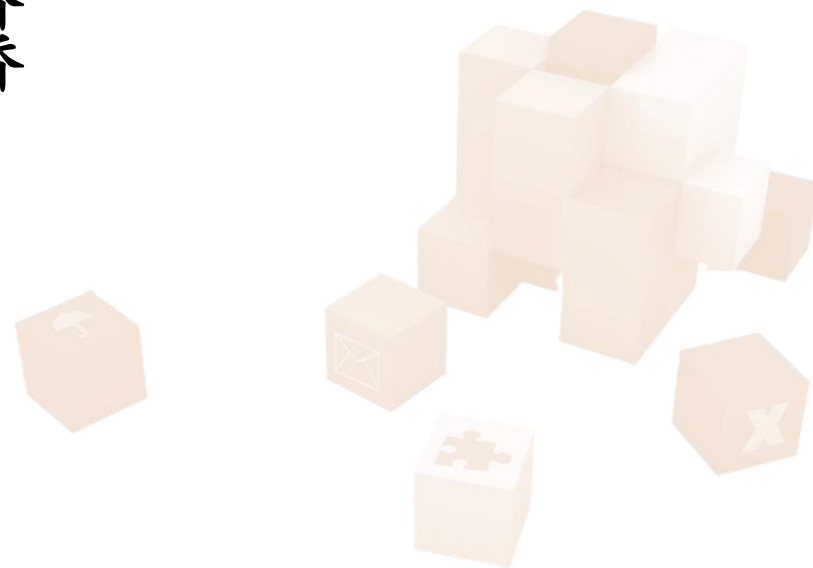
假如5个人过
独木桥



只能按上桥
的次序过桥



这里独木桥就是一个队列



3.2 队 列

3.2.1 队列的定义

队列的几个概念

- 进行插入的一端称做**队尾** (rear) 。
- 进行删除的一端称做**队首**或**队头** (front) 。
- 向队列中插入新元素称为**进队**或**入队**，新元素进队后就成为新的队尾元素。
- 从队列中删除元素称为**出队**或**离队**，元素出队后，其后继元素就成为队首元素。



3.2 队 列

3.2.1 队列的定义

队列抽象数据类型 = 逻辑结构 + 基本运算 (运算描述)

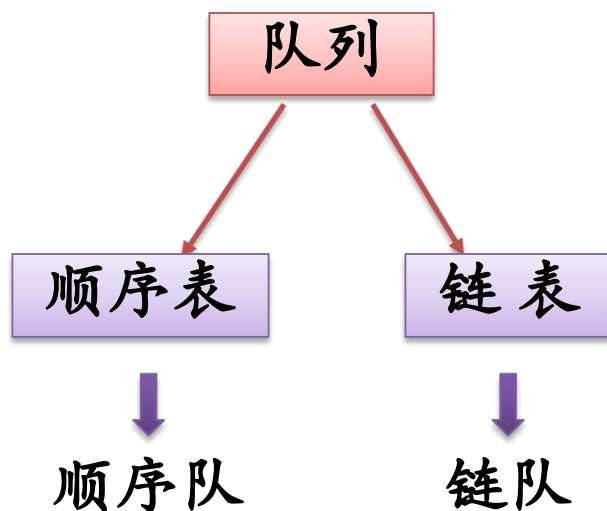
队列的基本运算如下:

- ① InitQueue(&q): 初始化队列。构造一个空队列q。
- ② DestroyQueue(&q): 销毁队列。释放队列q占用的存储空间。
- ③ QueueEmpty(q): 判断队列是否为空。若队列q为空, 则返回真; 否则返回假。
- ④ enQueue(&q, e): 进队列。将元素e进队作为队尾元素。
- ⑤ deQueue(&q, &e): 出队列。从队列q中出队一个元素, 并将其值赋给e。

3.2 队 列

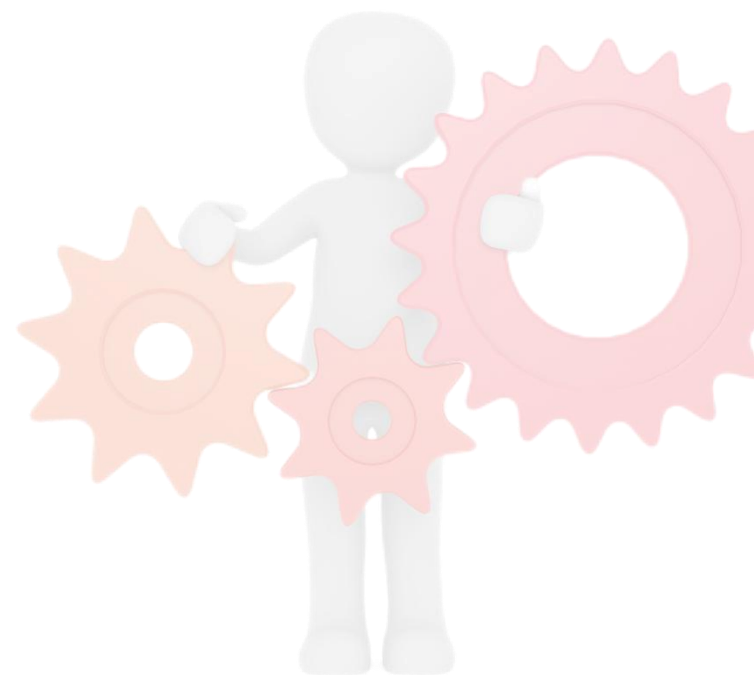
3.2.2 队列的顺序存储结构及其基本运算实现

队列中元素逻辑关系与线性表的相同，队列可以采用与线性表相同的存储结构。



逻辑结构

存储结构



3.2 队 列

3.2.2 队列的顺序存储结构及其基本运算实现

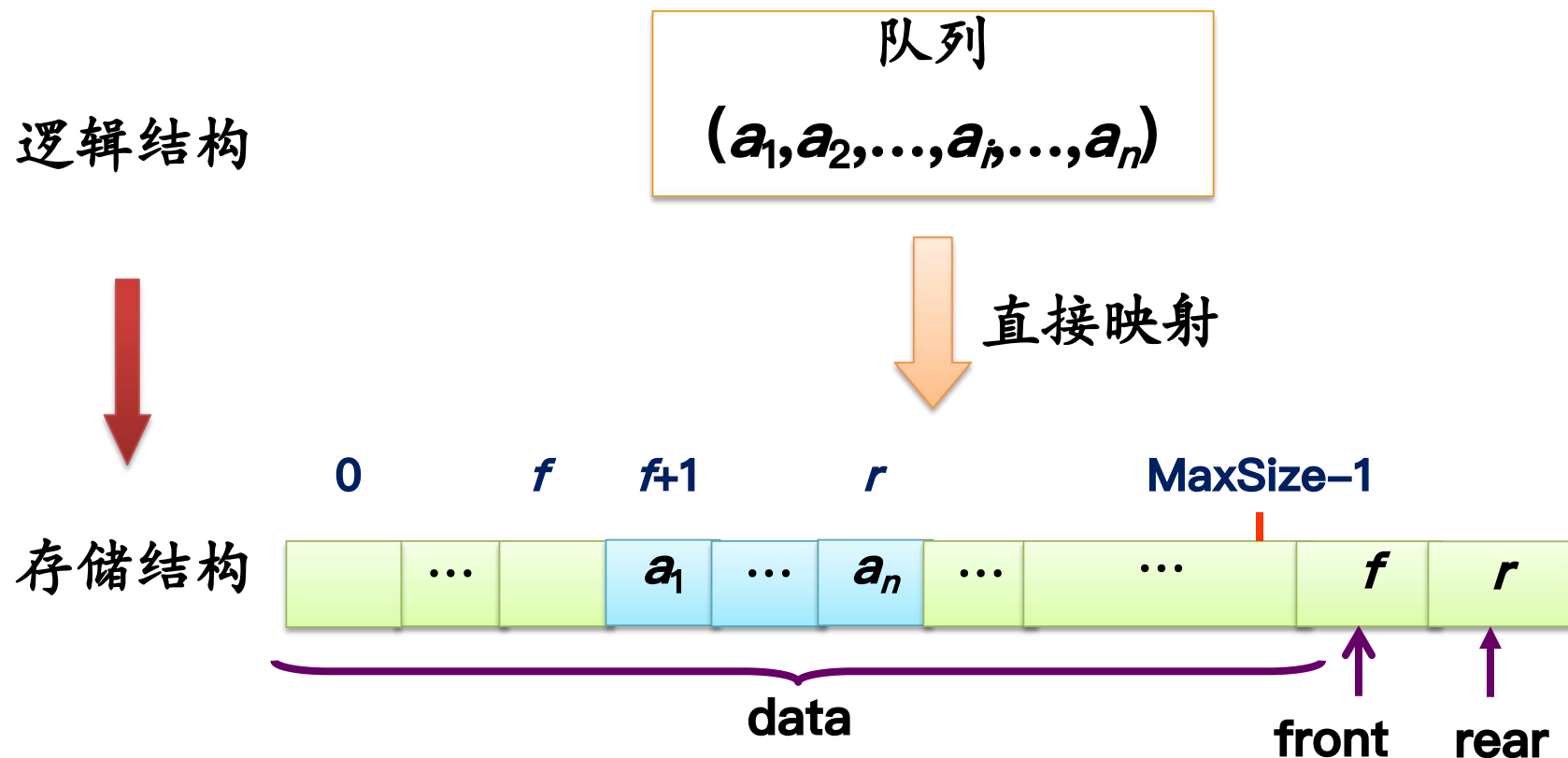
顺序队类型**SqQueue**声明如下：

```
typedef struct
{ ElemType data[MaxSize];
  int front, rear;      //队首和队尾指针
} SqQueue;
```

因为队列两端都在变化，所以需要两个指针来标识队列的状态。

3.2 队列

3.2.2 队列的顺序存储结构及其基本运算实现

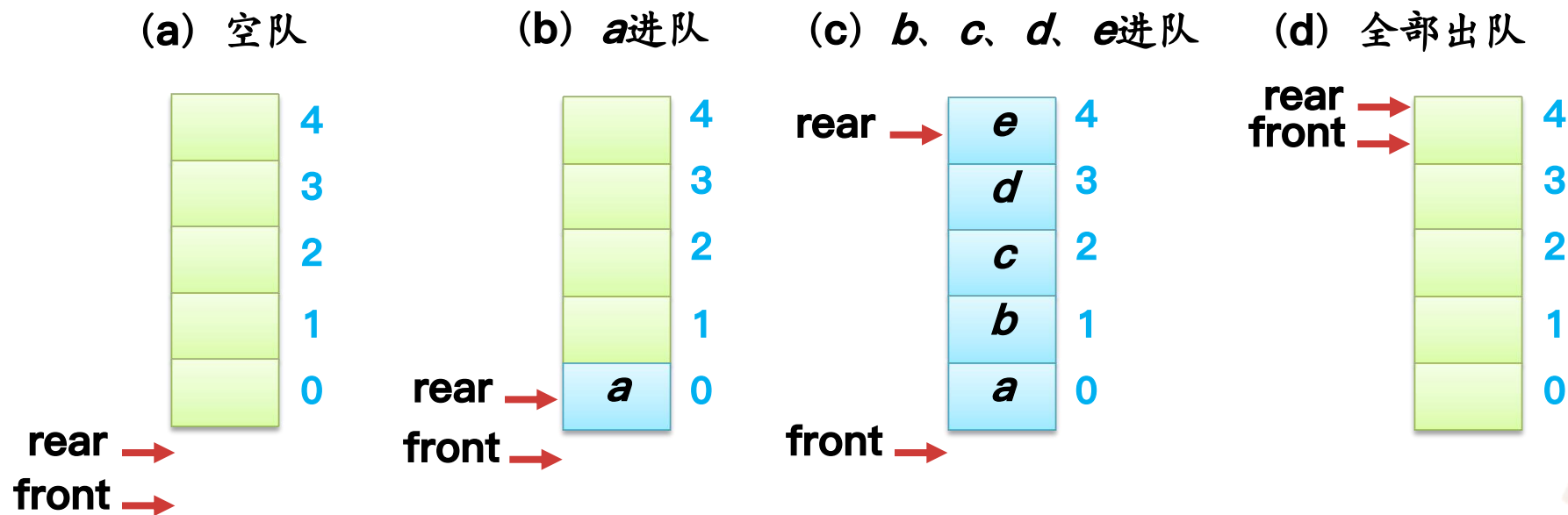


顺序队的示意图

3.2 队列

3.2.2 队列的顺序存储结构及其基本运算实现

例如: **MaxSize=5**



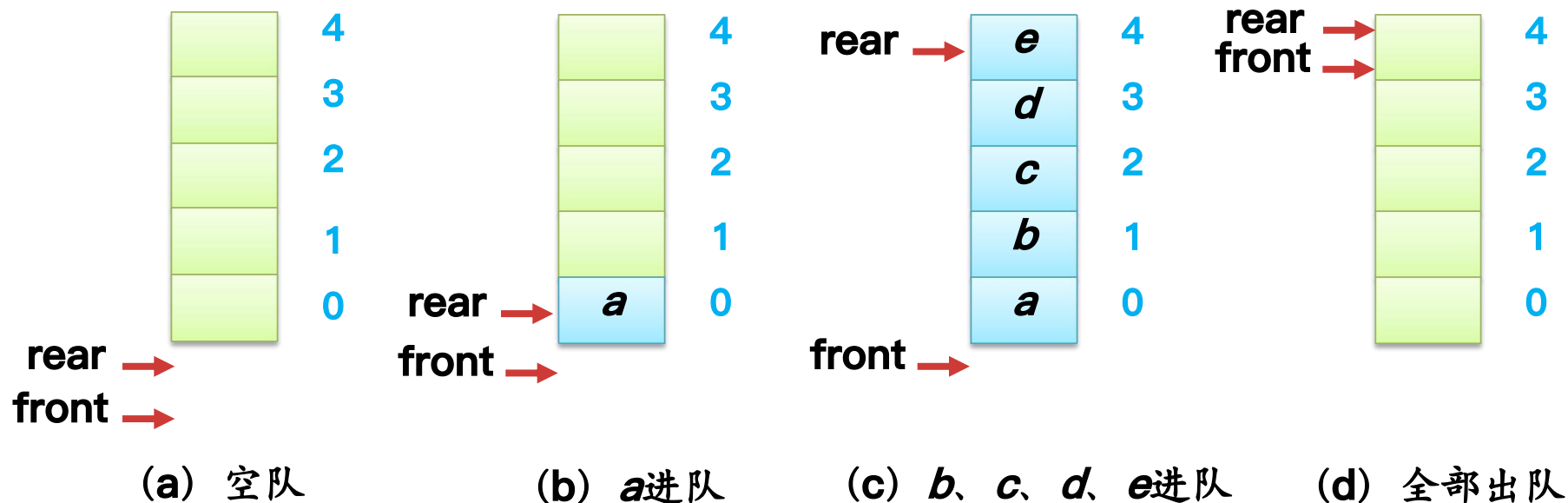
总结

- 约定 rear 总是指向队尾元素
- 元素进队, rear 增1
- 约定 front 指向当前队中队头元素的前一位置
- 元素出队, front 增1
- 当 $\text{rear}=\text{MaxSize}-1$ 时不能再进队

3.2 队列

3.2.2 队列的顺序存储结构及其基本运算实现

队列的各种状态



顺序队的4要素（初始时 $\text{front}=\text{rear}=-1$ ）：

- 队空条件： $\text{front} = \text{rear}$
- 队满条件： $\text{rear} = \text{MaxSize} - 1$
- 元素*e*进队： $\text{rear}++$; $\text{data}[\text{rear}]=\text{e}$;
- 元素*e*出队： $\text{front}++$; $\text{e}=\text{data}[\text{front}]$;

注意：

rear指向队尾元素；

front指向队头元素的前一个位置。

3.2 队列

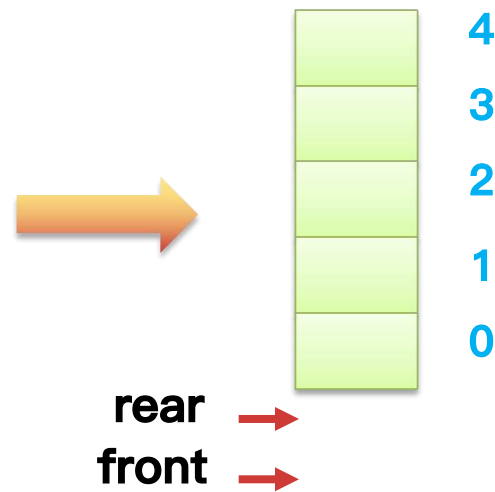
3.2.2 队列的顺序存储结构及其基本运算实现

1、顺序队中实现队列的基本运算算法

(1) 初始化队列InitQueue(q)

构造一个空队列q。将front和rear指针均设置成初始状态即-1值。

```
void InitQueue(SqQueue *&q)
{
    q=(SqQueue *)malloc (sizeof(SqQueue));
    q->front=q->rear=-1;
}
```



3.2 队 列

3.2.2 队列的顺序存储结构及其基本运算实现

(2) 销毁队列DestroyQueue(q)

释放队列q占用的存储空间。

```
void DestroyQueue(SqQueue *&q)
{
    free(q);
}
```



3.2 队 列

3.2.2 队列的顺序存储结构及其基本运算实现

(3) 判断队列是否为空 `QueueEmpty(q)`

若队列`q`满足`q->front==q->rear`条件，则返回`true`；否则返回`false`。

```
bool QueueEmpty(SqQueue *q)
{
    return(q->front==q->rear);
}
```

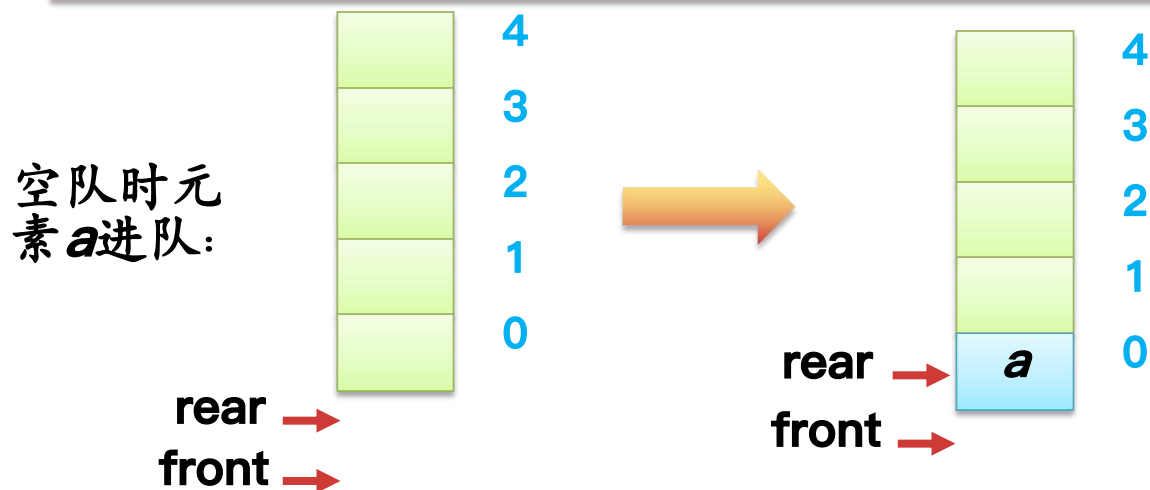
3.2 队 列

3.2.2 队列的顺序存储结构及其基本运算实现

(4) 进队列enQueue(q, e)

在队列不满的条件下，先将队尾指针rear循环增1，然后将元素添加到该位置。

```
bool enQueue(SqQueue *&q, ElemType e)
{   if (q->rear==MaxSize-1)           //队满上溢出
    return false;
    q->rear++;
    q->data[q->rear]=e;
    return true;
}
```



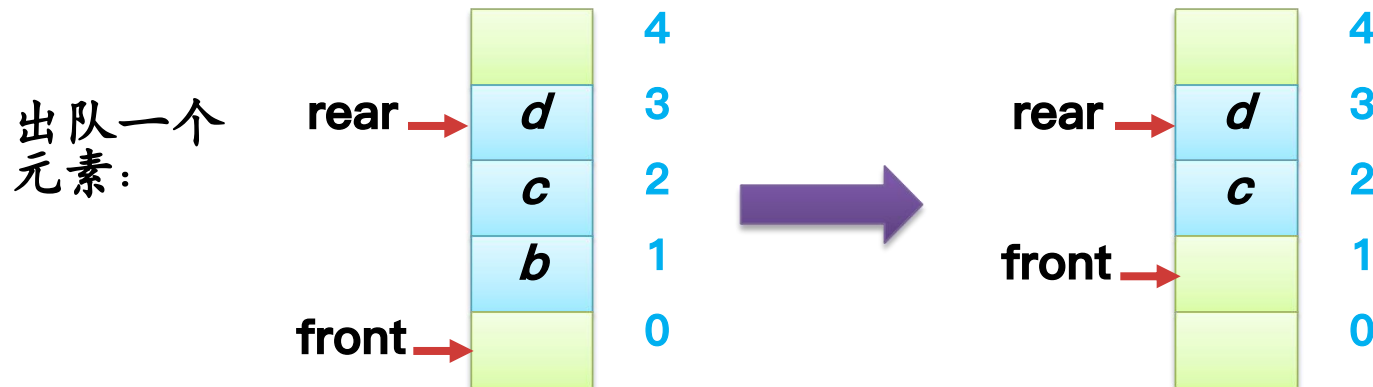
3.2 队列

3.2.2 队列的顺序存储结构及其基本运算实现

(5) 出队列deQueue(q, e)

在队列q不为空的条件下，将队首指针front循环增1，并将该位置的元素值赋给e。

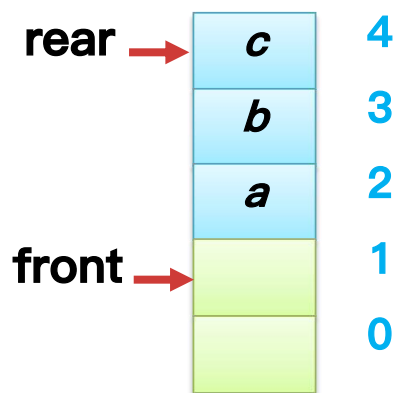
```
bool deQueue(SqQueue *&q, ElemType &e)
{ if (q->front==q->rear)           //队空下溢出
    return false;
  q->front++;
  e=q->data[q->front];
  return true;
}
```



3.2 队 列

3.2.2 队列的顺序存储结构及其基本运算实现

2、环形队列（或循环队列）中实现队列的基本运算



还有两个位置，为何不能进队？



这是因为采用 $\text{rear} == \text{MaxSize} - 1$ 作为队满条件的缺陷。当队满条件为真时，队中可能还有若干空位置。

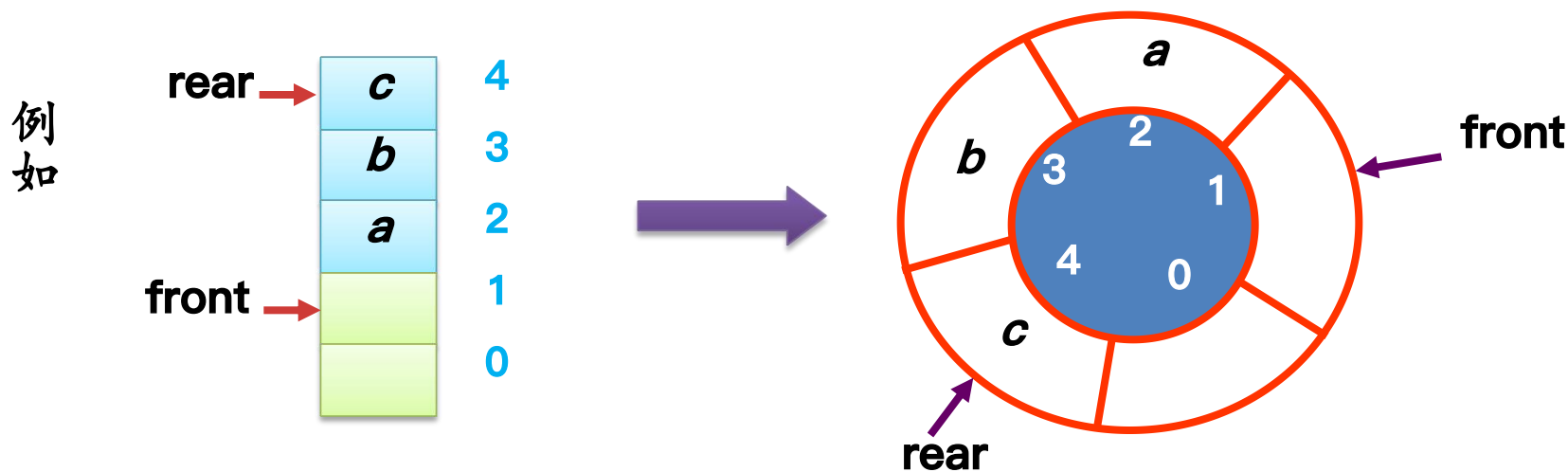
这种溢出并不是真正的溢出，称为 **假溢出**。

3.2 队列

3.2.2 队列的顺序存储结构及其基本运算实现

➤ 解决方案

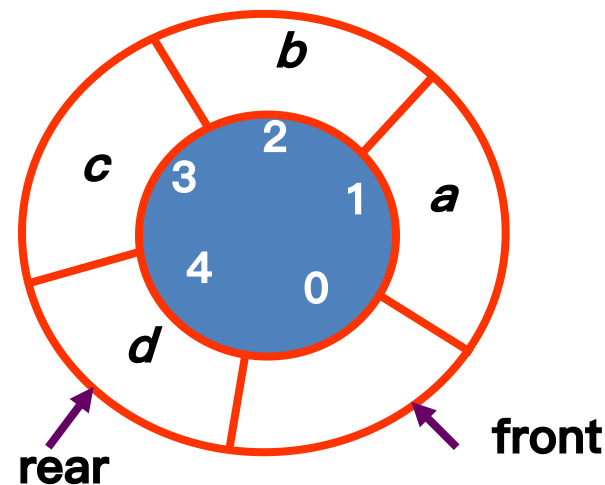
把数组的前端和后端连接起来，形成一个环形的顺序表，即把存储队列元素的表从逻辑上看成一个环，称为**环形队列**或**循环队列**。



3.2 队 列

3.2.2 队列的顺序存储结构及其基本运算实现

➤ 环形队列（循环队列）：



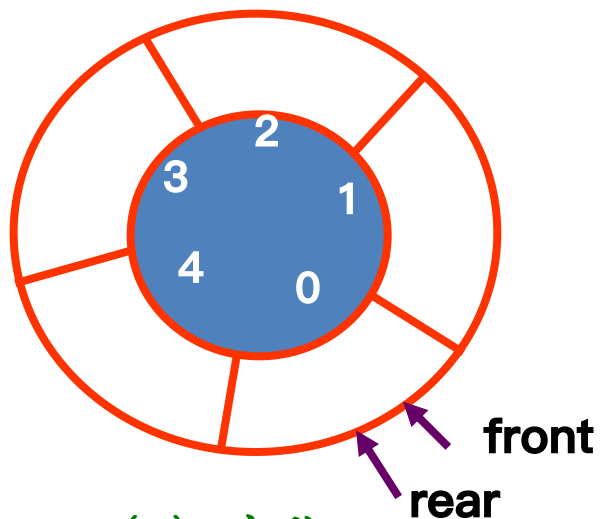
实际上内存地址一定是连续的，不可能是环形的，这里是通过逻辑方式实现环形队列，也就是将 $\text{rear}++$ 和 $\text{front}++$ 改为：

- $\text{rear} = (\text{rear} + 1) \% \text{MaxSize}$
- $\text{front} = (\text{front} + 1) \% \text{MaxSize}$

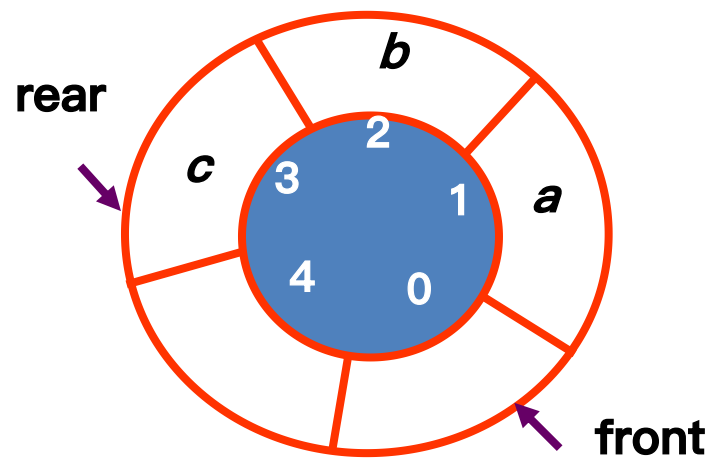


3.2 队列

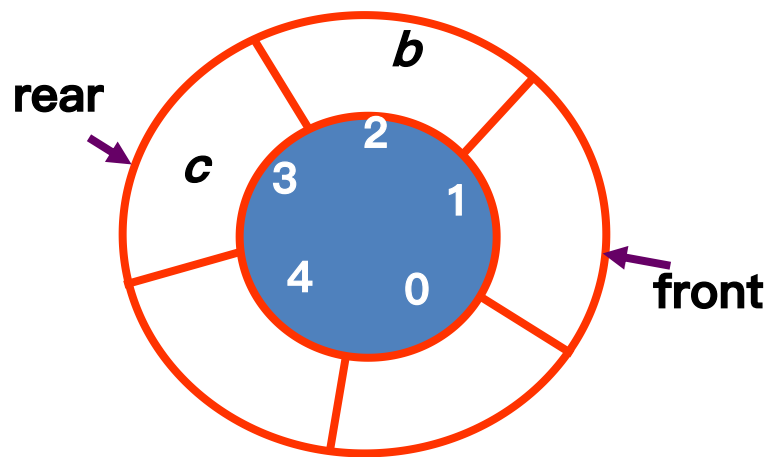
3.2.2 队列的顺序存储结构及其基本运算实现



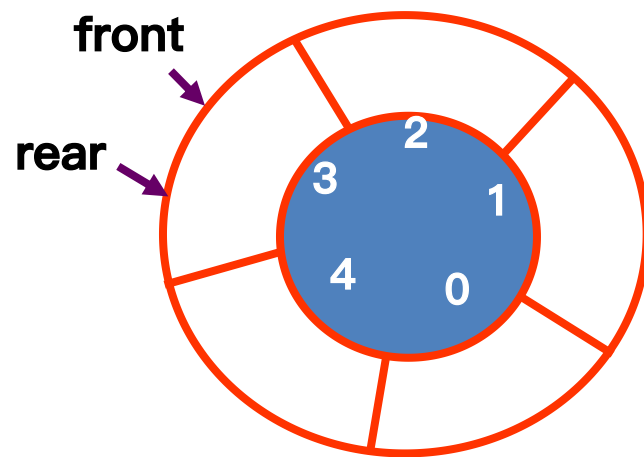
(a) 空队



(b) *a*、*b*、*c*进队



(c) 出队一次

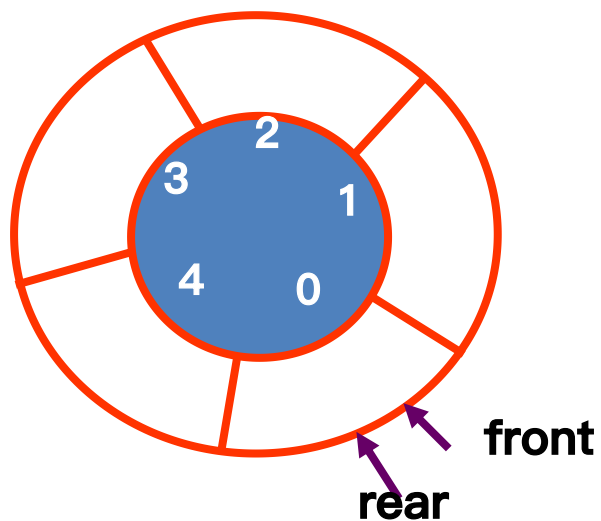


(d) 出队2次

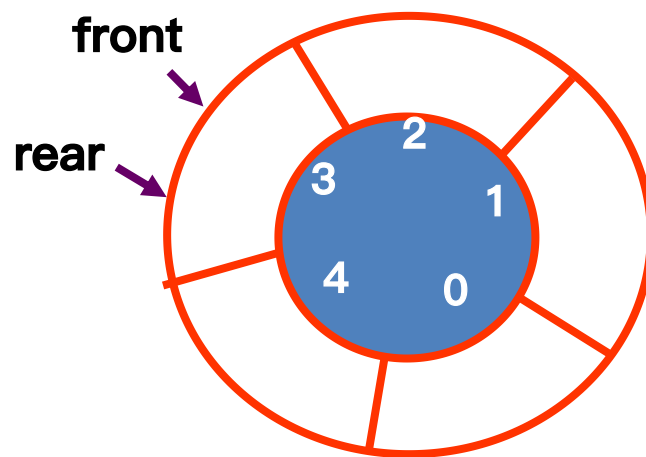
3.2 队列

3.2.2 队列的顺序存储结构及其基本运算实现

现在约定 $\text{rear}=\text{front}$ 为队空，以下两种情况都满足该条件：



初始状态



进队的所有元素均出队

那么如何设置队满的条件呢？

3.2 队 列

3.2.2 队列的顺序存储结构及其基本运算实现

让 $\text{rear} = \text{front}$ 为队空条件,

并约定队满条件为:

$$(\text{rear} + 1) \% \text{MaxSize} = \text{front}$$



进队一个元素时到达队头, 就认为队满了。这样做会少放一个元素, 牺牲一个元素没关系的



3.2 队 列

3.2.2 队列的顺序存储结构及其基本运算实现

环形队列的4要素:

- 队空条件: $\text{front} = \text{rear}$
- 队满条件: $(\text{rear} + 1) \% \text{MaxSize} = \text{front}$
- 进队e操作: $\text{rear} = (\text{rear} + 1) \% \text{MaxSize}$; 将e放在rear处
- 出队操作: $\text{front} = (\text{front} + 1) \% \text{MaxSize}$; 取出front处元素e;

在环形队列中, 实现队列的基本运算算法与非环形队列类似, 只是改为上述4要素即可。

3.2 队 列

3.2.2 队列的顺序存储结构及其基本运算实现

【例3.7】 对于环形队列来说，如果知道队头指针和队列中元素个数，则可以计算出队尾指针。也就是说，可以用队列中元素个数代替队尾指针。

设计出这种环形队列的初始化、入队、出队和判空算法。

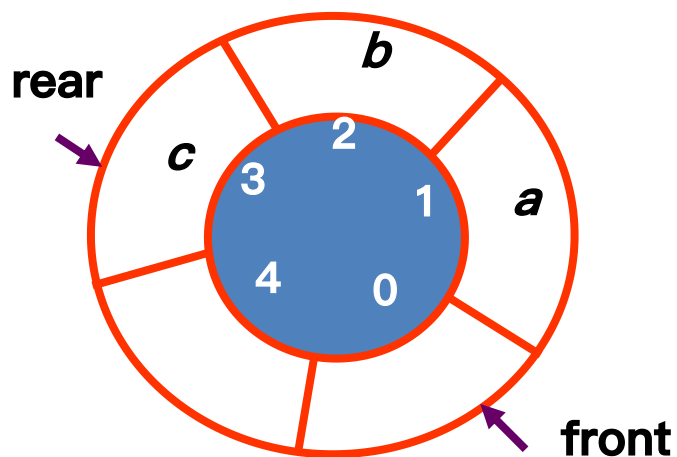


3.2 队列

3.2.2 队列的顺序存储结构及其基本运算实现

已知 **front**、**rear**，求队中元素个数 **count** = ?

MaxSize = 5

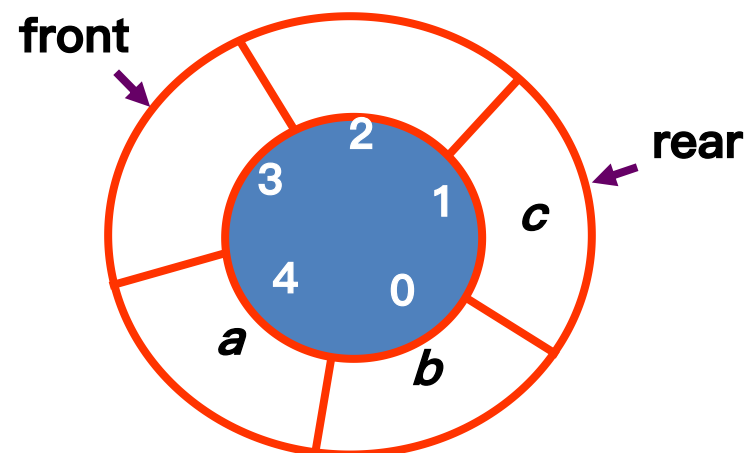


$$\text{count} = (3 - 0) = 3$$

$$\text{count} = \text{rear} - \text{front} \quad ?$$

$$\text{count} = (3 - 0 + \text{MaxSize}) = 8 \quad \times$$

$$\text{count} = (3 - 0 + \text{MaxSize}) \% \text{MaxSize} = 3 \quad \checkmark$$



$$\text{count} = (1 - 3) = -2 \quad \times$$

$$\text{count} = (1 - 3 + \text{MaxSize}) = 3$$

$$\text{count} = (1 - 3 + \text{MaxSize}) \% \text{MaxSize} = 3 \quad \checkmark$$

3.2 队 列

3.2.2 队列的顺序存储结构及其基本运算实现



已知**front**、**rear**，求队中元素个数**count**:

$$\text{count} = (\text{rear} - \text{front} + \text{MaxSize}) \% \text{MaxSize}$$

已知**front**、**count**，求**rear**:

$$\text{rear} = (\text{front} + \text{count}) \% \text{MaxSize}$$

已知**rear**、**count**，求**front**:

$$\text{front} = (\text{rear} - \text{count} + \text{MaxSize}) \% \text{MaxSize}$$

3.2 队 列

3.2.2 队列的顺序存储结构及其基本运算实现

解

依题意设计的环形队列类型如下：

```
typedef struct
{ ElemType data[MaxSize];

  int front;           //队头指针

  int count;           //队列中元素个数

} QuType;
```

3.2 队列

3.2.2 队列的顺序存储结构及其基本运算实现

该环形队列的4要素：

由front和count求出

- 队空条件： $\text{count} = 0$
- 队满条件： $\text{count} = \text{MaxSize}$
- 进队e操作： $\text{rear} = (\text{rear} + 1) \% \text{MaxSize}$ ；将e放在rear处
- 出队操作： $\text{front} = (\text{front} + 1) \% \text{MaxSize}$ ；取出front处元素e；

注意：这样的环形队列中最多可放置MaxSize个元素。

3.2 队 列

3.2.2 队列的顺序存储结构及其基本运算实现

对应的算法如下：

```
void InitQueue(QuType *&qu)    //初始化队运算算法
{
    qu=(QuType *)malloc(sizeof(QuType));

    qu->front=0;

    qu->count=0;

}
```

3.2 队 列

3.2.2 队列的顺序存储结构及其基本运算实现

[illegible]

3.2 队 列

3.2.2 队列的顺序存储结构及其基本运算实现

```
bool DeQueue(QuType *&qu, ElemType &x)           //出队运算算法  
{ if (qu->count==0)                                //队空下溢出  
    return false;  
  
    else  
    {        qu->front=(qu->front+1)%MaxSize;       //队头循环增1  
      x=qu->data[qu->front];  
      qu->count--;                                   //元素个数减1  
      return true;  
    }  
}
```

3.2 队 列

3.2.2 队列的顺序存储结构及其基本运算实现

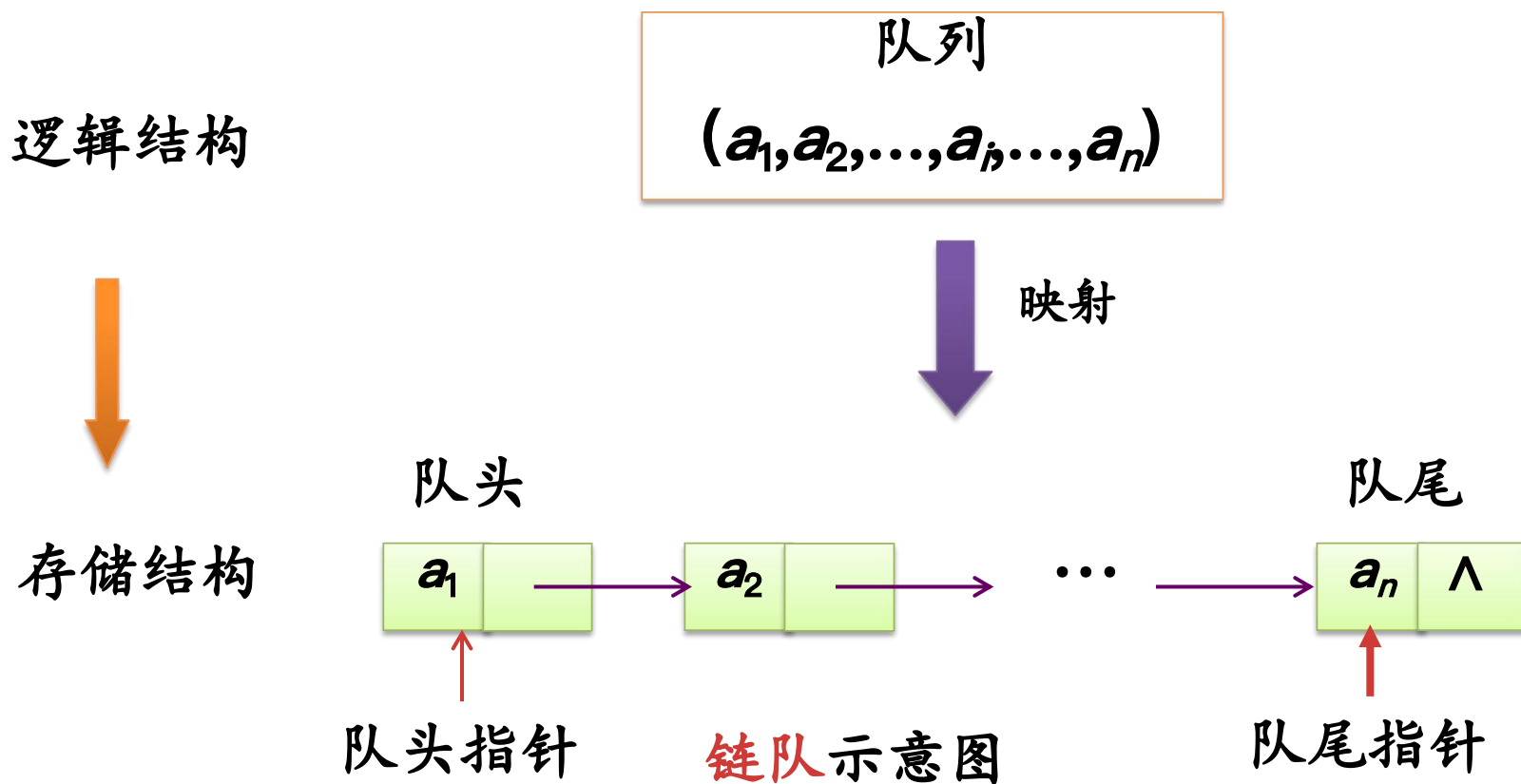
```
bool QueueEmpty(QuType *qu)    //判队空运算算法
{
    return(qu->count==0);
}
```



3.2 队 列

3.2.3 队列的链式存储结构及其基本运算的实现

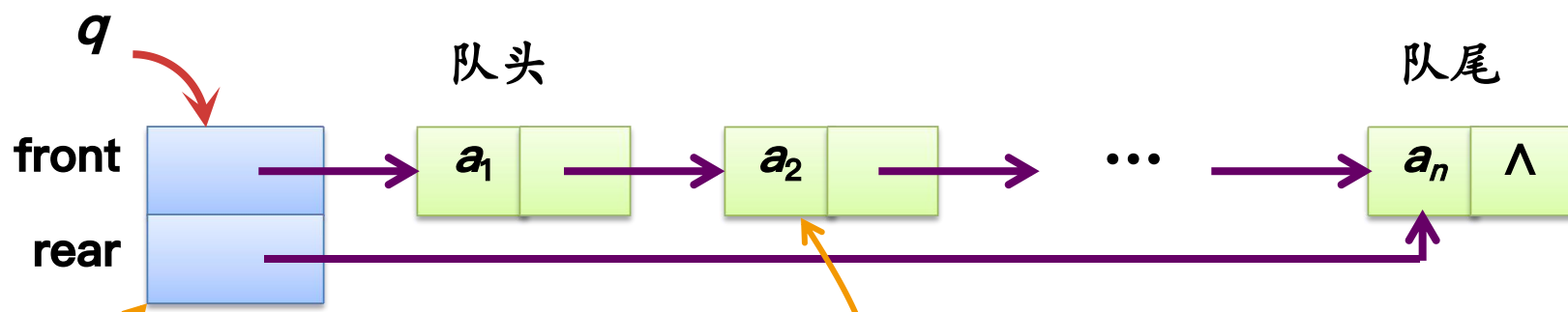
采用链表存储的队列称为**链队**，这里采用不带头结点的单链表实现。



3.2 队列

3.2.3 队列的链式存储结构及其基本运算的实现

通常将队头和队尾两个指针合起来:



链队组成:

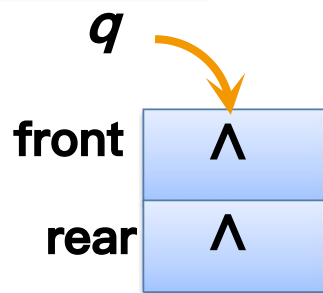
- (1) 存储队列元素的单链表结点
- (2) 指向队头和队尾指针的链队结点

3.2 队列

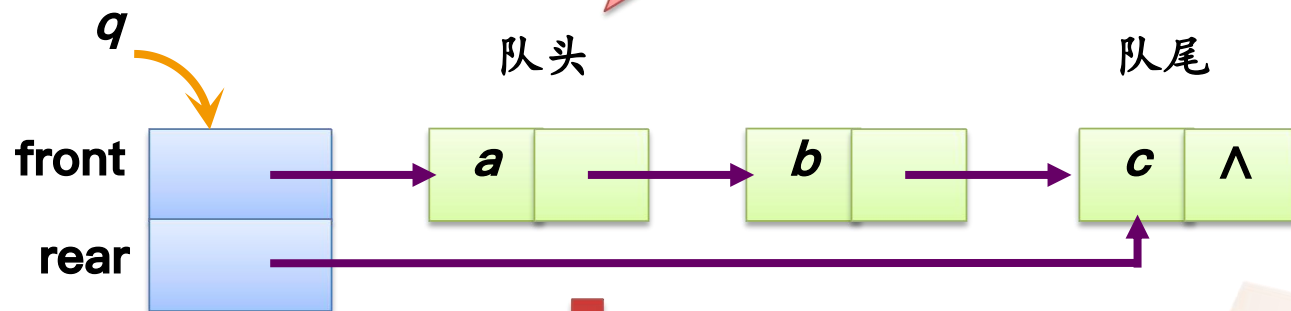
3.2.3 队列的链式存储结构及其基本运算的实现

链队的进队和出队操作演示

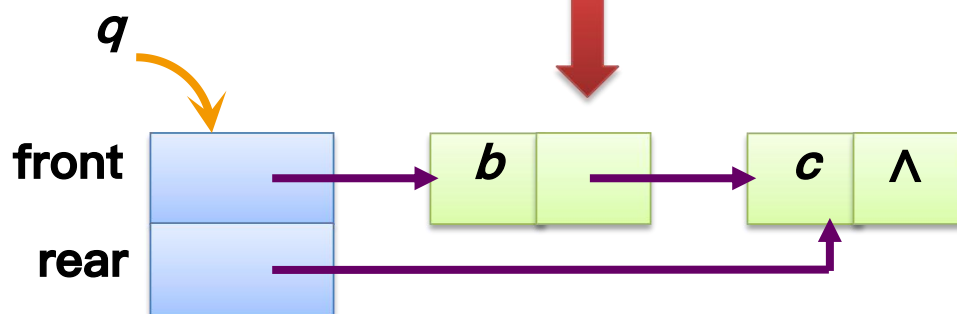
(a) 空队



(b) a 、 b 、 c 进队



(c) 出队一次



3.2 队 列

3.2.3 队列的链式存储结构及其基本运算的实现

单链表中数据结点类型**DataNode**声明如下：

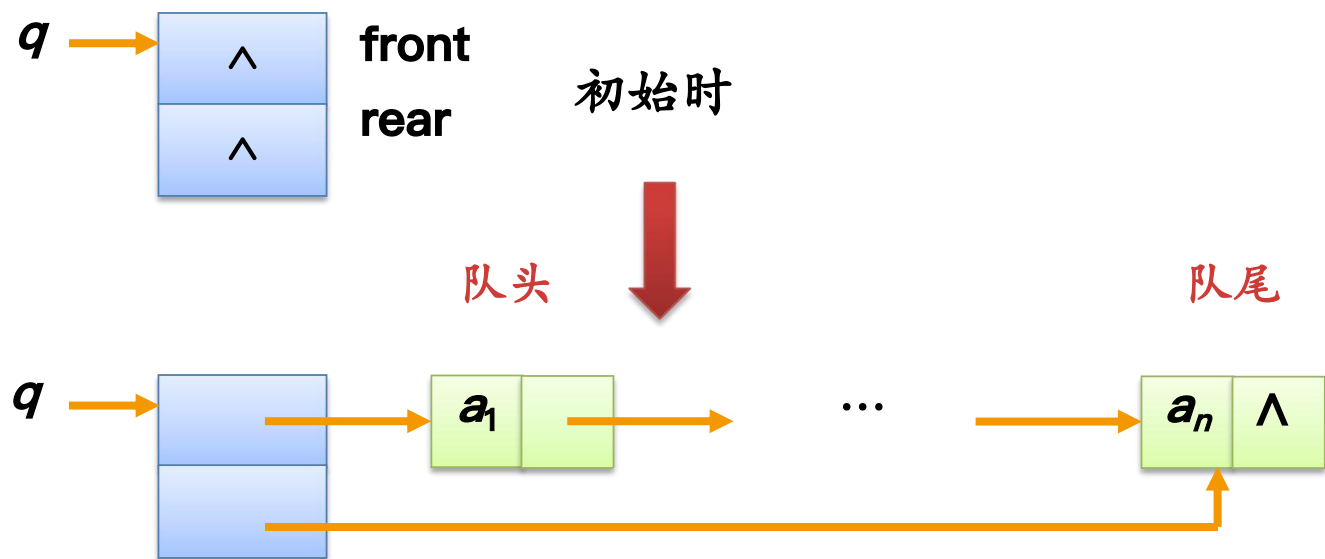
```
typedef struct qnode
{ ElemType data;          //数据元素
  struct qnode *next;
} DataNode;
```

链队结点类型**LinkQuNode**声明如下：

```
typedef struct
{ DataNode *front;        //指向单链表队头结点
  DataNode *rear;         //指向单链表队尾结点
} LinkQuNode;
```

3.2 队列

3.2.3 队列的链式存储结构及其基本运算的实现



- 队空条件: $\text{front}=\text{rear}=\text{NULL}$
- 队满条件: 不考虑
- 进队e操作: 将包含e的结点插入到单链表表尾
- 出队操作: 删除单链表首数据结点

3.2 队 列

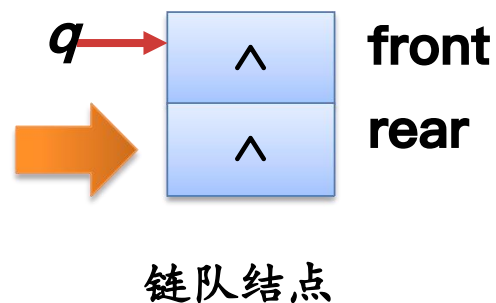
3.2.3 队列的链式存储结构及其基本运算的实现

在链队存储中，队列的基本运算算法。

(1) 初始化队列InitQueue(q)

构造一个空队列，即只创建一个链队头结点，其front和rear域均置为NULL，不创建数据元素结点。

```
void InitQueue(LinkQuNode *&q)
{ q=(LinkQuNode *)malloc(sizeof(LinkQuNode));
  q->front=q->rear=NULL;
}
```



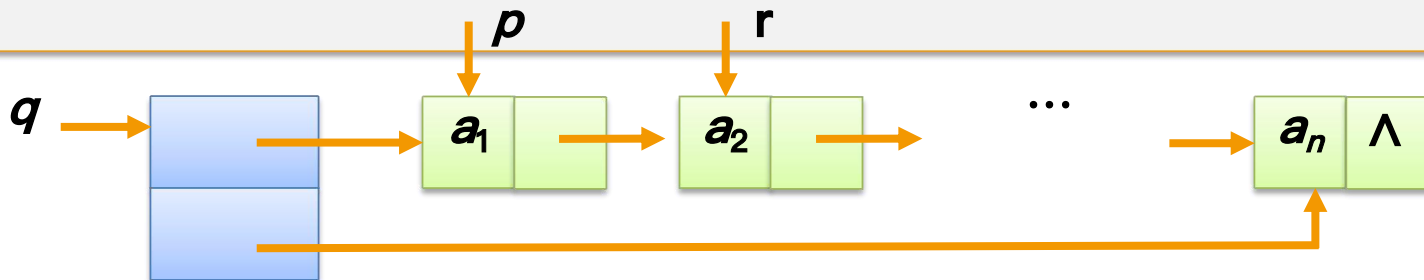
3.2 队列

3.2.3 队列的链式存储结构及其基本运算的实现

(2) 销毁队列DestroyQueue(q)

释放队列占用的存储空间，包括链队头结点和所有数据结点的存储空间。

```
void DestroyQueue(LinkQuNode *&q)
{
    DataNode *p=q->front, *r;    //p指向队头数据结点
    if (p!=NULL)                  //释放数据结点占用空间
    {
        r=p->next;
        while (r!=NULL)
        {
            free(p);
            p=r;r=p->next;
        }
    }
    free(p); free(q);             //释放链队结点占用空间
}
```



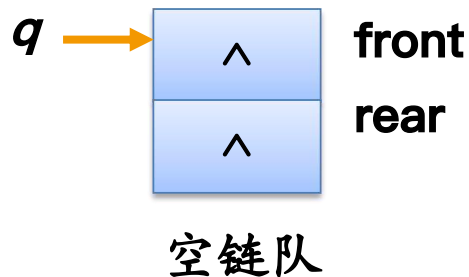
3.2 队 列

3.2.3 队列的链式存储结构及其基本运算的实现

(3) 判断队列是否为空 `QueueEmpty(q)`

若链队结点的rear域值为NULL，表示队列为空，返回true；否则返回false。

```
bool QueueEmpty(LinkQuNode *q)
{
    return(q->rear==NULL);
}
```



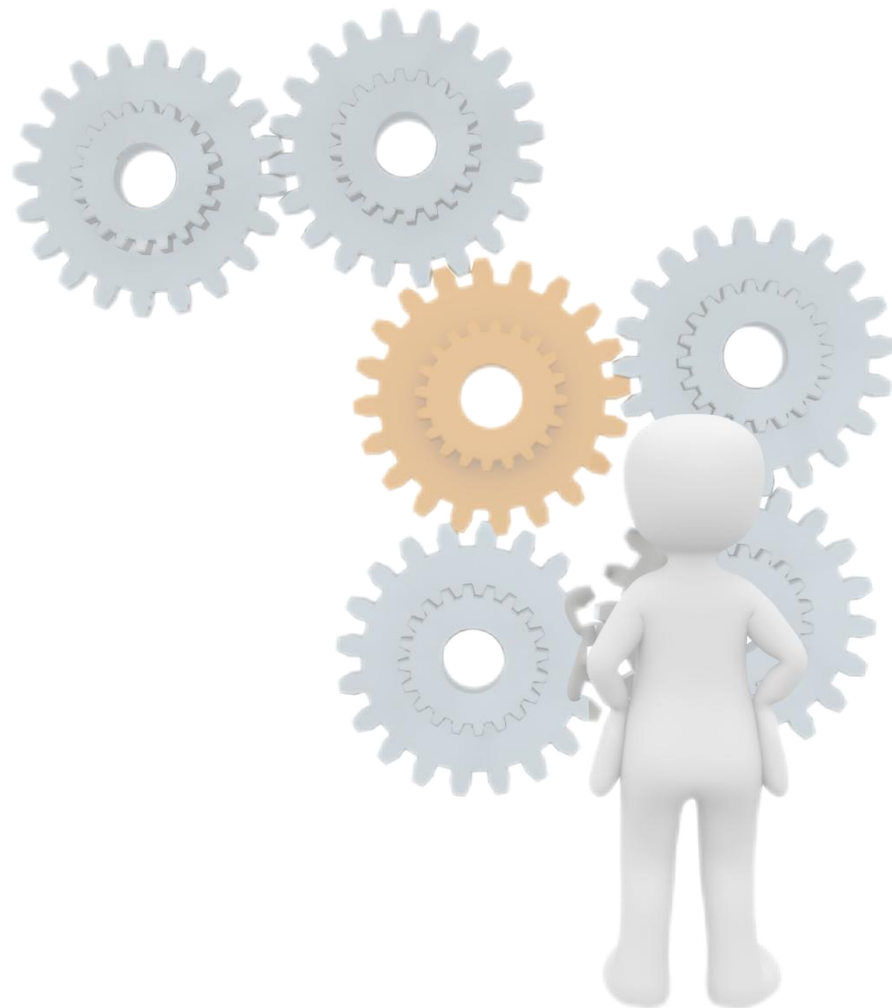
3.2 队 列

3.2.3 队列的链式存储结构及其基本运算的实现

(4) 进队enQueue(q, e)

考虑情况:

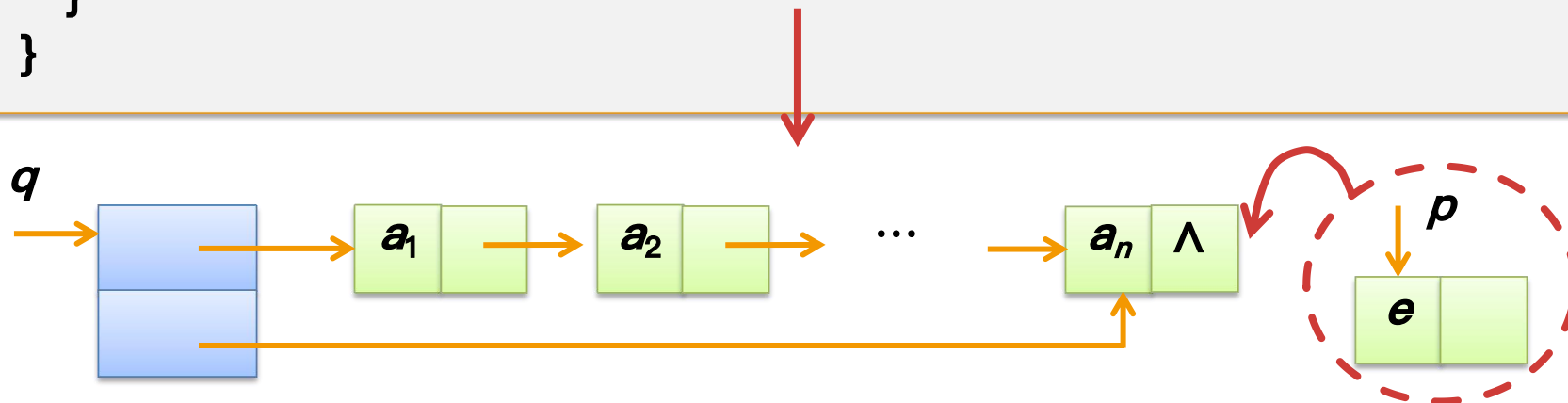
- 原队列为空
- 原队列非空



3.2 队列

3.2.3 队列的链式存储结构及其基本运算的实现

```
void enQueue(LinkQuNode *&q, ElemType e)
{
    DataNode *p;
    p=(DataNode *)malloc(sizeof(DataNode));
    p->data=e;
    p->next=NULL;
    if (q->rear==NULL) //若链队为空，新结点是队首结点又是队尾结点
        q->front=q->rear=p;
    else
    {
        q->rear->next=p; //将p结点链到队尾，并将rear指向它
        q->rear=p;
    }
}
```



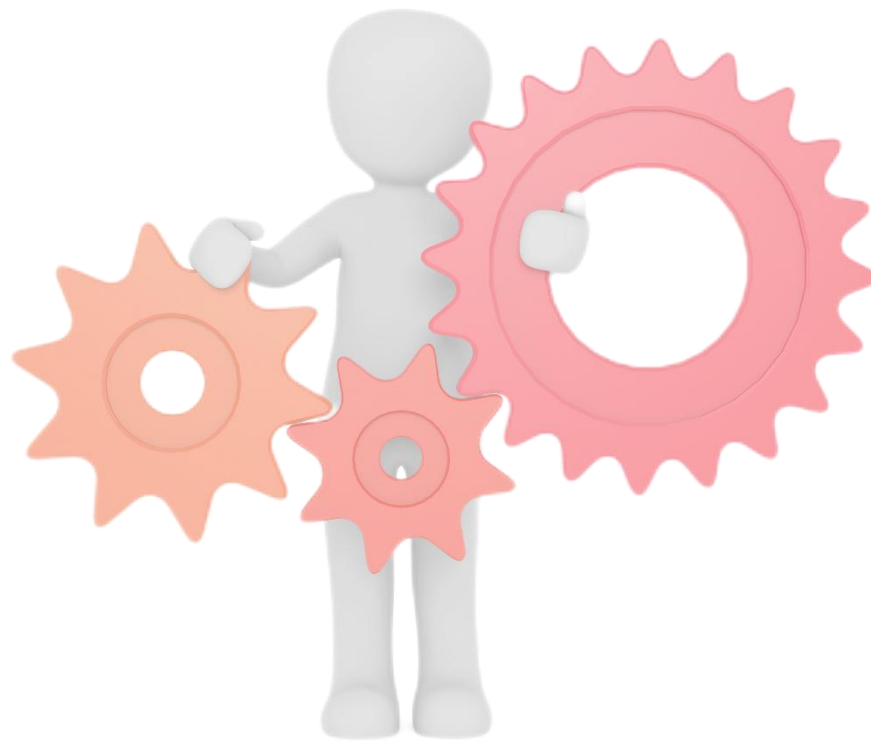
3.2 队 列

3.2.3 队列的链式存储结构及其基本运算的实现

(5) 出队deQueue(q, e)

考虑情况:

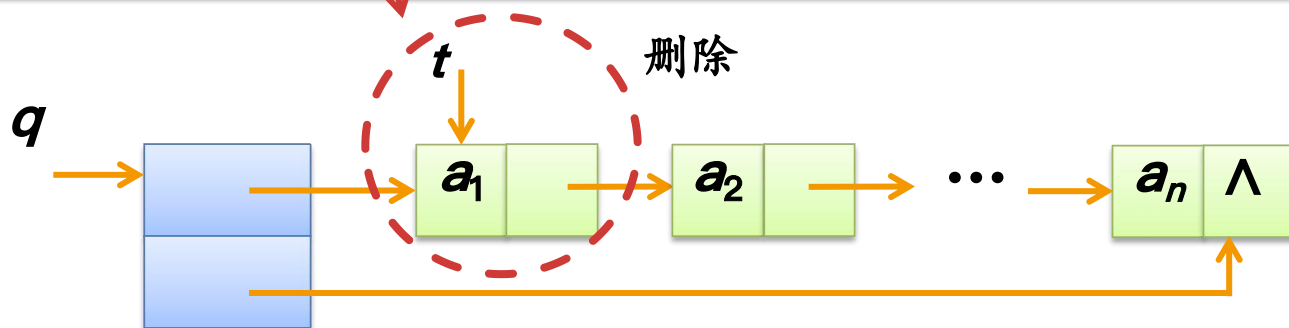
- 原队列为空
- 原队列只有一个结点
- 其他情况



3.2 队列

3.2.3 队列的链式存储结构及其基本运算的实现

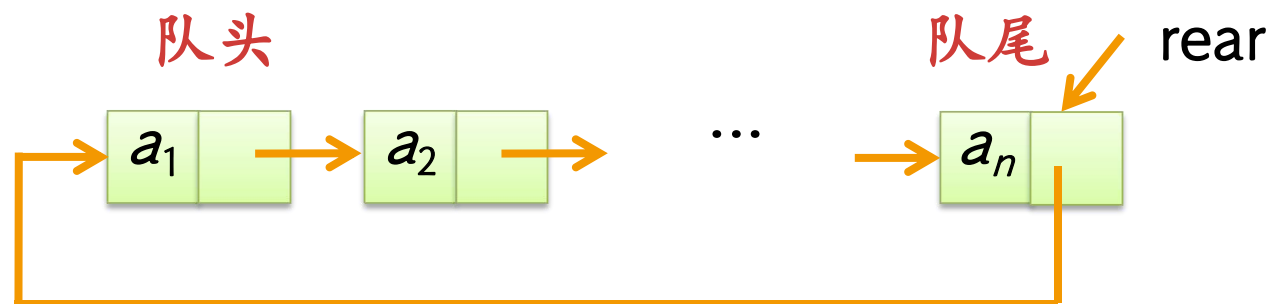
```
bool deQueue(LinkQuNode *&q, ElemType &e)
{
    DataNode *t;
    if (q->rear==NULL) return false;           //队列为空
    t=q->front;                                 //t指向第一个数据结点
    if (q->front==q->rear)                       //队列中只有一个结点时
        q->front=q->rear=NULL;
    else                                         //队列中有多个结点时
        q->front=q->front->next;
    e=t->data;
    free(t);
    return true;
}
```



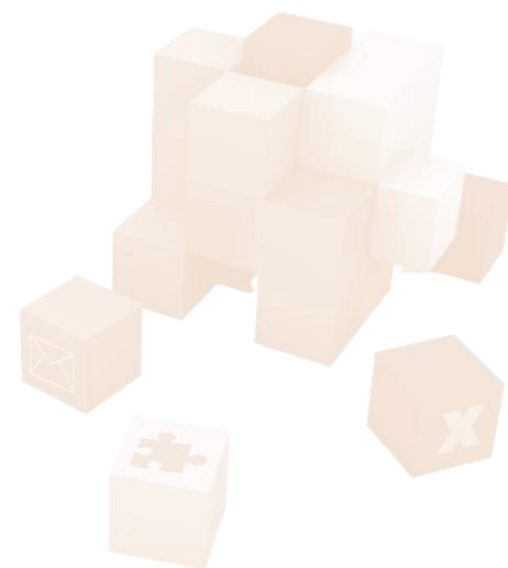
3.2 队列

3.2.3 队列的链式存储结构及其基本运算的实现

【例3.8】 采用一个**不带头结点**只有一个尾结点指针**rear**的循环单链表存储队列，设计队列的初始化、进队和出队等算法。

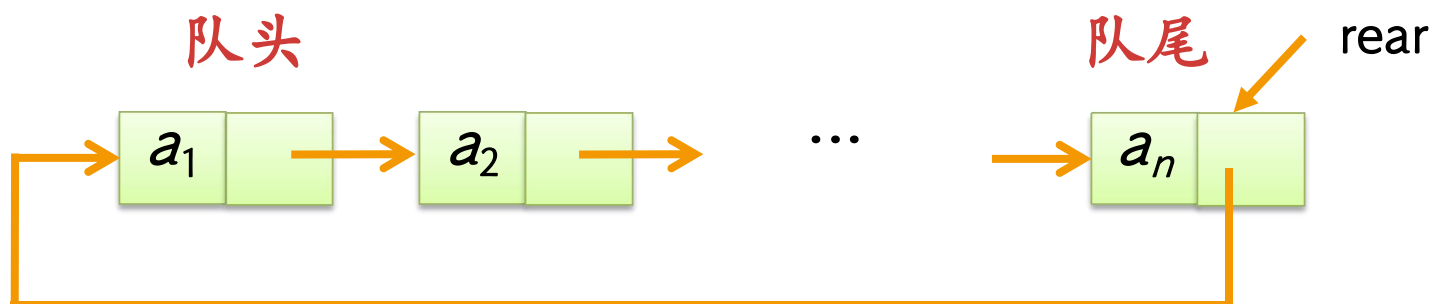


这样的链队通过尾结点指针**rear**唯一标识。



3.2 队列

3.2.3 队列的链式存储结构及其基本运算的实现



这样的链队通过尾结点指针rear唯一标识。

链队的4要素：

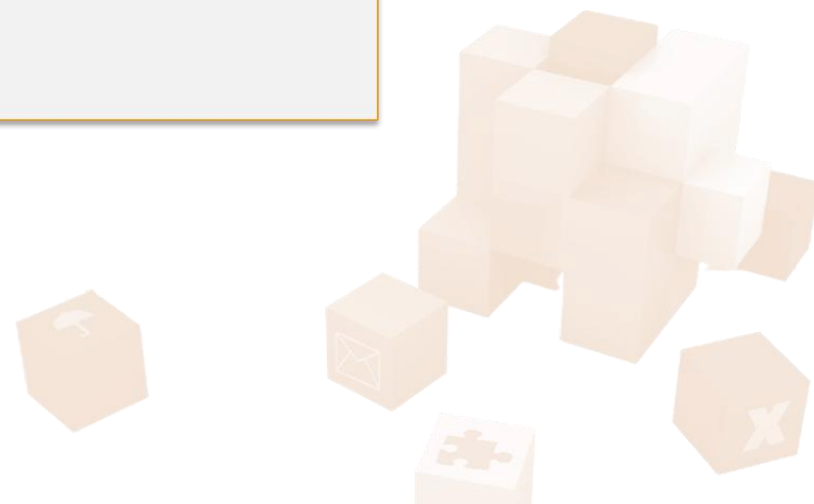
- 队空条件： $\text{rear} = \text{NULL}$
- 队满条件： 不考虑
- 进队e操作： 将包含e的结点插入到单链表表尾
- 出队操作： 删除单链表首结点

3.2 队 列

3.2.3 队列的链式存储结构及其基本运算的实现

```
void InitQueue(LinkList *&rear) //初始化队运算算法
{
    rear=NULL;
}

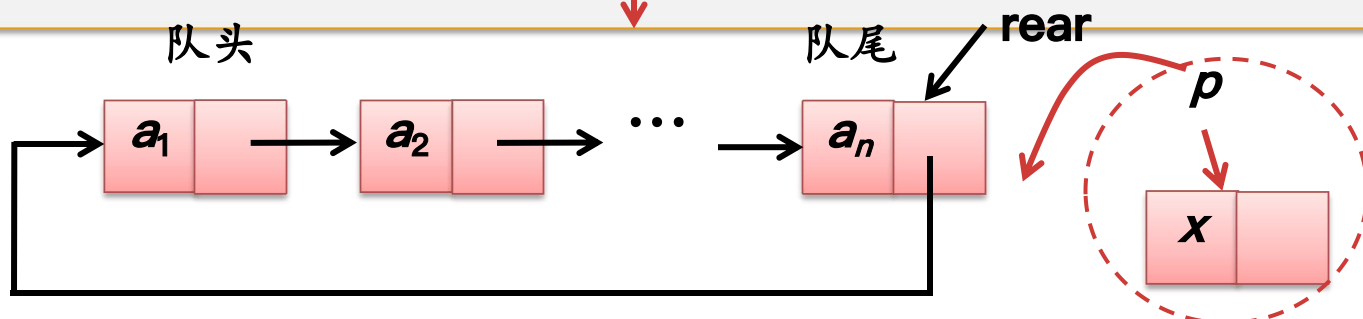
bool queueEmpty(LinkList *rear) //判队空运算算法
{
    return(rear==NULL);
}
```



3.2 队列

3.2.3 队列的链式存储结构及其基本运算的实现

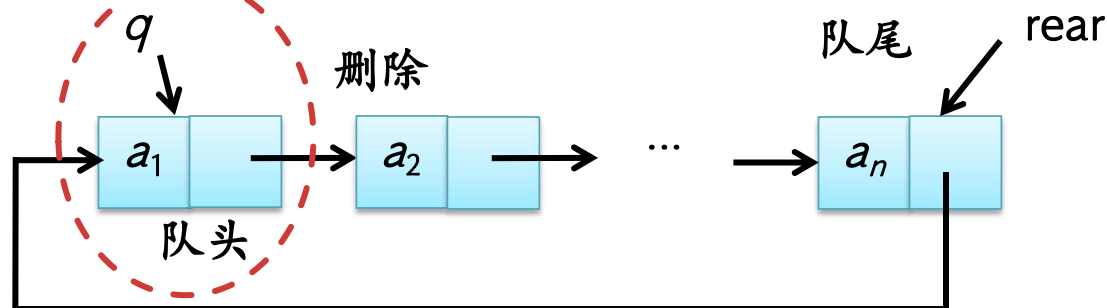
```
void enQueue(LinkList *&rear, ElemType x) //进队运算算法
{
    LinkList *p;
    p=(LinkList *)malloc(sizeof(LinkList)); //创建新结点
    p->data=x;
    if (rear==NULL) //原链队为空
    {
        p->next=p; //构成循环链表
        rear=p;
    }
    else
    {
        p->next=rear->next; //将p结点插入rear结点之后
        rear->next=p;
        rear=p; //让rear指向新插入的结点
    }
}
```



3.2 队列

3.2.3 队列的链式存储结构及其基本运算的实现

```
bool deQueue(LinkList *&rear, ElemType &x) //出队运算算法
{
    LinkList *q;
    if (rear==NULL) return false;           //队空
    else if (rear->next==rear)              //原队只有一个结点
    {
        x=rear->data;
        free(rear);
        rear=NULL;
    }
    else                                     //原队有两个或以上的结点
    {
        q=rear->next;
        x=q->data;
        rear->next=q->next;
        free(q);
    }
    return true;
}
```



3.2 队列

3.2.4 队列的应用

1. 求解报数问题

➤ 问题描述

设有 n 个人站成一排，从左向右的编号分别为 $1 \sim n$ ，现在从左往右报数“1, 2, 1, 2, ...”，数到“1”的人出列，数到“2”的立即站到队伍的最右端。报数过程反复进行，直到 n 个人都出列为止。要求给出他们的出列顺序。

例如，当 $n=8$ 时，初始序列为

1 2 3 4 5 6 7 8

则出列顺序为

1 3 5 7 2 6 4 8

3.2 队列

3.2.4 队列的应用

➤ 数据组织

用一个队列解决出队问题，由于这里不需要使用已经出队后的元素，所以采用环形队列。

- 
- 
- 环形队列中出队的元素可能被覆盖。
 - 非环形队列中出队的元素不可能被覆盖。

- 如果利用队列求解时，希望结束后再利用所有出队的元素进一步查找结果，使用**非环形队列**，如迷宫问题。
- 否则，使用**环形队列**。

3.2 队列

3.2.4 队列的应用

➤ 运算算法设计

算法思想：先将 n 个人的编号进队，然后反复执行如下操作，直到队列为空：

- ① 出队一个的元素，输出其编号——**报数为1的人出列。**
- ② 若队列不空，则再出队一个元素，并将刚出列的元素进队——**报数为2的人站到队伍的最右端即队尾。**

3.2 队 列

3.2.5 双端队列

所谓**双端队列**是指两端都可以进行进队和出队操作的队列，将队列的两端分别称为前端和后端，两端都可以入队和出队。

其元素的逻辑结构仍是线性结构。

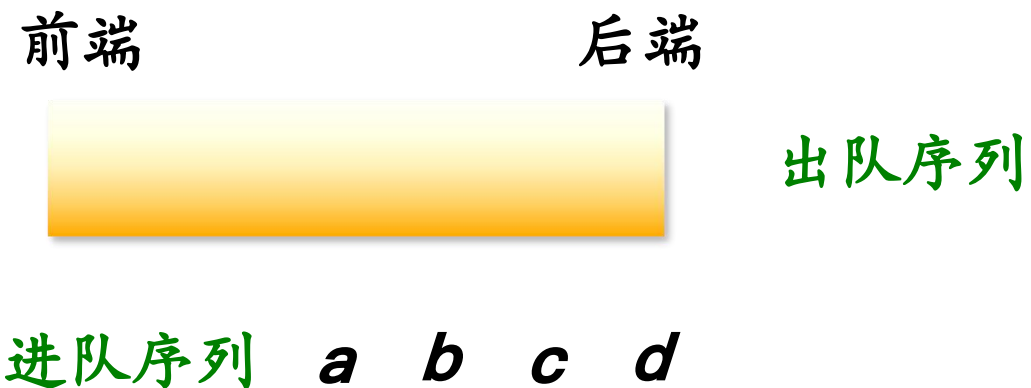


3.2 队 列

3.2.5 双端队列

例如，有 a 、 b 、 c 、 d 元素进队，能否产生 $dcab$ 的出队序列呢？

答案是肯定的。



3.2 队 列

3.2.5 双端队列

输出受限的双端队列，即一个端点允许插入和删除，另一个端点只允许插入的双端队列。



3.2 队列

3.2.5 双端队列

【例3.10】 某队列允许在两端进行入队操作，但仅允许在一端进行出队操作，若 a 、 b 、 c 、 d 、 e 元素进队，则不可能得到的顺序是（ ）。

A. $bacde$ B. $dbace$ C. $dbcae$ D. $ecbad$

说明：本题为2010年全国考研题。

答案为C。

前端

后端



出队序列

进队序列 a b c d e

选项C操作： a 后端进， b 前端进，因 d 未出，此时只能进队， c 怎么进都不可能在 b 、 a 之间。

3.2 队 列

3.2.5 双端队列

输入受限的双端队列，即一个端点允许插入和删除，另一个端点只允许删除的双端队列。



第3章 栈和队列

提纲

CONTENTS

小结

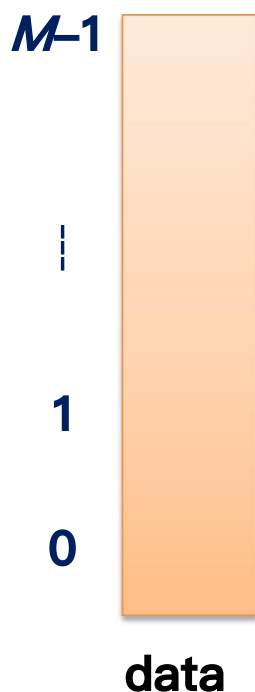
小结——栈

① 先进后出表。

② 一个大小为 n 的顺序栈，最多只能进行 n 次进栈操作吗？

错误： 可以进行任意多次进栈操作。但最多只能进行**连续** n 次进栈操作。

③ 顺序栈只能将栈底设置在 $\text{data}[0]$ 端吗？



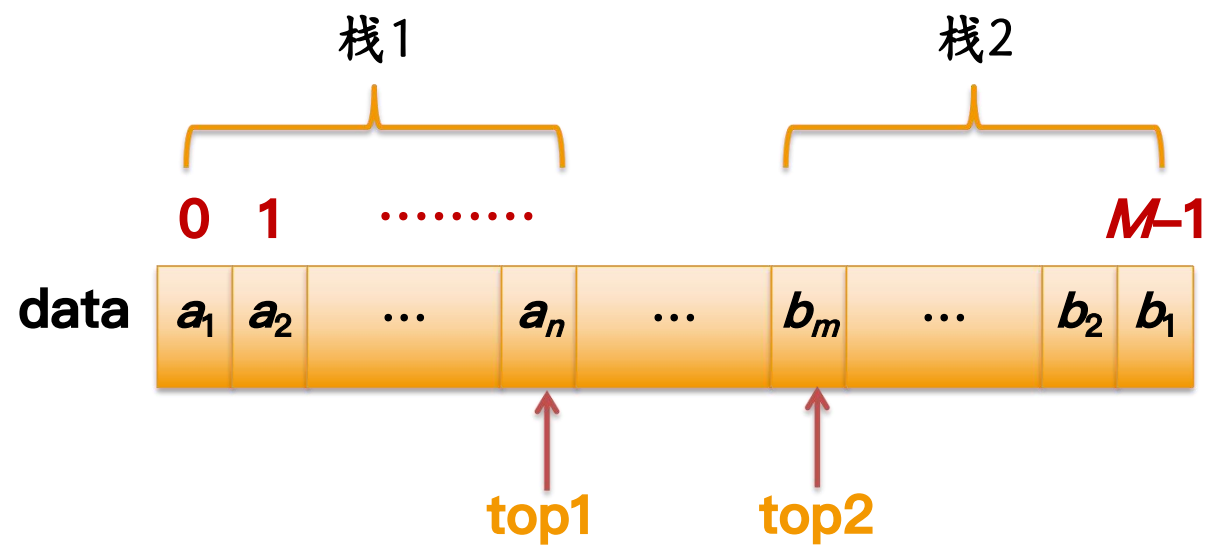
可以将栈底设置在任意一端，但不能设置在中间。

将栈底设置在 $\text{data}[M-1]$ 端的设计：

- **初始化：** $\text{top} = M$
- **栈空：** $\text{top} == M$
- **栈满：** $\text{top} == 0$ （最小下标）
- **元素 e 进栈：** $\text{top}--$; $\text{data}[\text{top}] = e$
- **出栈：** $e = \text{data}[\text{top}]; \text{top}++$

小结——栈

④ 共享栈



小结——队列

① 先进先出表。1, 2, 3, ..., n通过一个队列的出队序列个数?

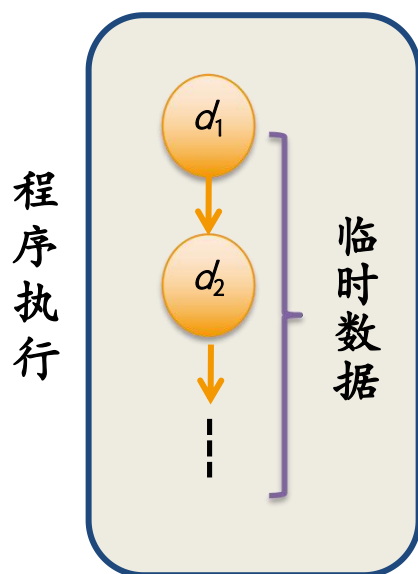
只有一个: 即1, 2, 3, ..., n

小结——队列

② 环形队列解决了假溢出问题，任何情况下都使用环形队列吗？

- 采用环形队列时，进队的元素可能被覆盖。
- 如果需要用队列中全部进队的元素进一步求解问题，应该采用非环形队列。如用队列求解迷宫路径！

小结——栈和队列的应用



保存 d_1 、 d_2 、...

- 先产生的数据后处理—**栈**（后进先出表）
- 先产生的数据先处理—**队列**（先进先出表）